

Determining software quality using code analysis metrics

Yevheniia Kataieva and Martin Nemec

Abstract—This paper addresses the evaluation of software quality through the use of code analysis metrics. Software quality is critical to ensuring functionality, reliability, and maintainability, with complexity serving as a fundamental factor affecting these aspects. The study identifies common "code smells" structural issues within code that indicate potential weaknesses and categorizes them into dispensable elements, bloaters, coupling issues, and more. Object-oriented metrics such as Lines of Code (LOC), Cyclomatic Complexity (CC), and Number of Methods per Class (NOM) are explored for their roles in measuring and managing code complexity. Furthermore, the study proposes new metrics, including Relative Complexity Index (RCI) and Interaction Complexity (IC), designed to enhance the detection of code smells and improve maintainability. Implementation results using a Maven plugin for Spring Boot projects demonstrate the practicality of these metrics in generating detailed reports for software developers.

Keywords - software quality, code smells, code analysis metrics, Relative Complexity Index, Interaction Complexity, maintainability

I. INTRODUCTION

The success of any project is determined by its ability to meet the needs of the consumer, so ensuring a high level of quality is a necessary task of any production, including software engineering. Software quality is a complex concept. For each of the quality components, a set of metrics can be called that determine the quality of the software product. A software metric is a measure that allows you to obtain a numerical value of some property of the software or its specifications, as well as a method for calculating it. Complexity is an important factor on which other software quality parameters depend, such as accuracy, correctness, reliability, and ease of maintenance. The existence of methods and algorithms for automatically calculating software complexity metrics using software tools allows you to get a comprehensive formal report on software quality in a short time.

Software becomes less complex with the more skillfully it is developed. With decreasing complexity, the program is easier to navigate and write new code, the risk of making mistakes is lower, and the chances of discovering something new are higher. Reduced complexity significantly reduces the time of its development and the ability to accompany it.

This work was supported by the Slovak Research and Development Agency under the Contract no. APVV-23-0408.

Yevheniia Kataieva is with Slovak University of Technology in Bratislava, Slovakia (e-mail: yevheniia.kataieva@stuba.sk)

Martin Nemec is with Slovak University of Technology in Bratislava, Slovakia (e-mail: xnemecm1@stuba.sk)

979-8-3315-1764-9/25/\$31.00 ©2025 European Union

Several tools and techniques for code quality analysis have been developed, particularly for Java programming. Tools effectively detect general code smells and vulnerabilities, they often lack the detailed contextual analysis needed to evaluate relationships between classes and methods. The metrics proposed in this study, such as the Relative Complexity Index (RCI) and Interaction Complexity (IC), aim to fill this gap by providing a more granular assessment of maintainability risks, particularly in object-oriented and Spring Boot projects.

II. UNDERSTANDING CODE QUALITY AND ITS IMPORTANCE

Code issues, commonly termed as "code smells," represent specific signs or structures in code that indicate potential problems or weaknesses. They are not bugs or errors or specific problems that cannot run the code. The code will compile still with code smells. For instance, code duplication, convoluted 'if' conditions, improper names of variables or unused code blocks are examples of such issues. These code deficiencies, while seemingly minor at first place, can lead to serious long-term problems such as low application performance or compromised code maintainability[1].

This section briefly categorizes code smells and their implications for software quality:

- **Dispensable Code Smells:** Unnecessary code such as duplicates, lazy classes, or dead code [2].
- **Bloaters:** Excessively large methods, classes, or parameter lists that reduce readability and maintainability[3].
- **Object-Orientation Abusers:** Poor design decisions such as repeated switch statements and lack of polymorphism [4].
- **Couplers:** High coupling between classes or modules, leading to interdependencies that complicate changes[5].
- **Change Preventers:** Structures that make modifications time-consuming and error-prone[6].
- **Testing Smells:** Inefficient test structures indirectly impacting code quality[7].

Although this section has addressed various categories of code smells, their impact on code quality, and specific examples, the primary focus of this research is the identification of complex methods.

Object-oriented(OO) code metrics are used to measure the quality of the code. The main use of these metrics is to manage code complexity, prevent code smells, and improve

the quality of the code. The terminology says that they should be used for object-oriented code, but many OO metrics can be used for procedural code too. Every metric has a clear definition of what aspects of the code quality they measure[8].

The following are widely adopted OO metrics:

- **Lines of Code (LOC):** Measures software size by counting the number of lines in the source code[9].
- **Number of Methods per Class (NOM):** Counts the methods in a class to indicate complexity[10].
- **Cyclomatic Complexity (CC):** Directly measures the number of linearly independent paths through a method, class, or program. The formula for calculating the cyclomatic complexity is as follows:

$$CC = E - N + 2P \quad (1)$$

Where:

- E is the number of edges in the flow graph,
- N is the number of nodes in the flow graph,
- P is the number of connected components.

A higher CC value indicates greater complexity. The table below shows the risk evaluations based on CC:

TABLE I
CYCLOMATIC COMPLEXITY RISK EVALUATION[21]

Cyclomatic complexity	Risk evaluation
1-10	Simple program, low risk
11-20	Moderate complexity, moderate risk
21-50	High complexity, high risk
51+	Untestable program, very high risk

This threshold for CC is also applied in the Interaction Complexity (IC) metric, where a value greater than 10 indicates significant complexity.

- **Weighted Methods per Class (WMC):** Measures the total complexity of all methods in a class, calculated as the sum of their individual complexities. This metric provides insight into how the method design affects the overall complexity of the class [10].

Many other metrics, such as Depth of Inheritance Tree (DIT), Number of Children (NOC), and Lack of Cohesion in Methods (LCOM), are also used in software quality analysis. These metrics provide a broader perspective on the structure, design, and maintainability of code.

III. PROPOSING NEW METRICS FOR CODE SMELLS DETECTION

In this section, new metrics will be introduced that can be used to measure the quality of the code. The new metrics will be based on existing metrics and will be used to detect code smells for complex methods faster. This study will focus on the Cyclomatic Complexity (CC) metric, FANIN and FANOUT metrics. The CC metric was described in the previous chapter. The FANIN metric is used to measure the number of methods that call a method, and the FANOUT metric is used to measure the number of methods that a method calls. If method is called 10 times, FANIN will be

10 and if method calls 3 other methods, the FANOUT is set to 3. The Relative Complexity Index (RCI) metric, the Relative Complexity Index Max Tolerance (RCIM) metric, and the Interaction Complexity (IC) metric are proposed. Listed metrics will be used to detect code smell - complex methods in the code. The proposed metrics will provide more detailed insight into the complexity of the code and help identify potential code smells more effectively.

A. Relative Complexity Index (RCI)

The Relative Complexity Index (RCI) metric focuses on comparing the complexity of a specific method to the average complexity of its containing class. The theoretical motivation for choosing Cyclomatic Complexity (CC) as the foundational metric lies in its established use as a complexity indicator, particularly in object-oriented software design. The metric will be calculated for each method in each class in the project. To calculate the RCI metric, the average CC of the class and the CC of the actual method need to be calculated. The formula that is used to calculate the RCI metric is:

$$RCI = \frac{CC_{method}}{CC_{avgClass}}, \quad CC_{avgClass} = \frac{\sum_{i=1}^n CC_i}{n} \quad (2)$$

where CC method is the cyclomatic complexity of the method and CCavgClass is the average cyclomatic complexity of the class. The final value of the RCI metric will be a decimal number. The RCI metric will be used to evaluate the complexity of the method compared to the other methods in the class. The main goal of the RCI metric is to detect complex methods compared to the other methods in the class, which may need to be refactored and may cause maintenance issues in the future. Because if a class has many methods and one of them is very complex, it can be hard to maintain the class in the future. However, the RCI metric on its own is not very useful, because now, as it is, it is just a decimal number. Another metric, the Relative Complexity Index - Max Tolerance (RCIM), will be defined for comparison with the RCI metric.

B. Relative Complexity Index - Max Tolerance (RCIM)

Next metric that is defined is the Relative Complexity Index - Max Tolerance (RCIM). The RCIM metric will be used as a comparable metric to the RCI metric. As the name suggests, the RCIM metric will be used to define the maximum tolerance level of the RCI metric. The RCIM metric will be calculated for each class in the project. To calculate the RCIM metric, we need to calculate the average RCI metric of all methods in the class and calculate the standard deviation of the RCI metric. The formula that will be used to calculate the RCIavg metric is:

$$RCI_{avgclass} = \frac{\sum_{i=1}^n RCI_i}{n} \quad (3)$$

where n is the number of methods in a class, and RCI_i is the RCI metric of the i-th method in a class. The standard deviation will be used to calculate the tolerance level and

to evaluate the complexity of the method. The standard deviation will be calculated by using the formula:

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (RCI_i - RCI_{avgClass})^2}{n}} \quad (4)$$

where n is the number of methods in a class, and RCI_i is the RCI metric of the i -th method in a class. The final formula used to calculate the RCIM metric is as follows:

$$RCIM = RCI_{avgClass} + k * \sigma \quad (5)$$

where $RCI_{avgClass}$ is the average RCI metric of all methods in a class, σ is the standard deviation of the RCI metric, and k is the parameter that tells us how strict we want to be when evaluating the complexity of the method.

The application of the proposed metrics is illustrated using an example class containing 8 methods. The first seven methods are simple, while the last one is complex. Each detection process is illustrated separately. The process for detecting methods with high complexity compared to the class average using RCI and RCIM is as follows:

- **Step 1: Calculation of RCI** For the seven simple methods assume that each method has $CC = 1$, one method has $CC = 9$ and the average cyclomatic complexity of the class is $CC_{avgClass} = 2$ (computed as the sum of the CC values divided by the number of methods). The RCI values for each simple method is calculated as follows:

$$RCI = \frac{1}{2} = 0.5$$

For the complex method with $CC = 9$:

$$RCI = \frac{9}{2} = 4.5$$

- **Step 2: Calculation of RCIM**

The average RCI for the class is calculated as:

$$RCI_{avgClass} = \frac{(0.5 \cdot 7) + 4.5}{8} = 1$$

The standard deviation (σ) is:

$$\sigma = \sqrt{\frac{\sum (RCI_i - RCI_{avgClass})^2}{n}} \approx 1.323$$

For $k = 2$, the RCIM threshold is:

$$RCIM = 1 + 2 \cdot 1.323 = 3.646$$

- **Step 3: Comparison of RCI and RCIM**

The RCI of the complex method (4.5) exceeds the RCIM threshold (3.646), indicating that the method is a lot more complex than the other methods in the class.

C. Interaction Complexity (IC)

The Interaction Complexity (IC) metric is the final defined metric. To find possible code smells in the code more quickly, the IC metric will be used. This will be accomplished by considering the FANIN and FANOUT measurements in addition to the CC metric. This is because a method with high cyclomatic complexity and a large number of methods calling

it could lead to serious maintainability problems down the road. The complexity of the code increases the likelihood that programmers make mistakes and makes it harder to maintain. The CC metric will serve as the foundation for the IC metric. The FANIN and FANOUT measures are also required. The IC metric will be computed using the following formula:

$$IC = CC + \log(FANIN + 1) + 0.5 * \log(FANOUT + 1) \quad (6)$$

where CC is the cyclomatic complexity of the method, FANIN is the number of methods that call the method, and FANOUT is the number of methods that the method calls. The logarithm function is applied to normalize the values of the FANIN and FANOUT metrics, with FANOUT further multiplied by the coefficient 0.5, because the FANIN metric should have a greater impact on the IC metric than the FANOUT metric. This is because if a method is called by many methods, it can be harder to maintain the method in the future, as mentioned earlier. The use of the logarithm function in the IC metric is crucial to avoid exceedingly large resulting values, especially when the FAN-IN and FAN-OUT values are high. Without logarithmic scaling, metrics for methods with extensive connections could produce disproportionately high values, rendering comparisons less meaningful. The logarithmic function ensures that the incremental impact decreases as the number of calling or called methods grows, capturing the idea that it is more impactful if a method is called by 1 versus 10 methods, compared to 100 versus 120.

Visualization of IC Process

The process for detecting methods with high interaction complexity using IC is as follows:

- **Step 1: Calculation of IC**

For the method with $CC = 9$, $FANIN = 10$, and $FANOUT = 2$:

$$IC = 9 + \log(10 + 1) + 0.5 \cdot \log(2 + 1)$$

$$IC \approx 9 + 1.04 + 0.238 \approx 10.28$$

- **Step 2: Evaluation of IC**

The IC value of the method (10.28) exceeds the defined threshold of 10 (as outlined in the chapter 'Understanding Code Quality and Its Importance' under the metric definition of cyclic complexity), indicating that the method is complex and may require refactoring.

IV. IMPLEMENTATION

The developed application is a Maven plugin optimized for Spring Boot projects. It can be easily integrated into a project by adding it to the pom.xml file. Once Maven commands like mvn clean install are executed, the plugin automatically runs and performs the following actions:

- Scans the project for Spring Boot applications,
- Analyzes defined code metrics (e.g., LOC, CC, NOM, etc.).
- Identifies code smells based on the defined thresholds.
- Generates a report in the form of a HTML file.

The parsing of the project is done by library named JavaParser, and it use AST (Abstract Syntax Tree) to analyze the code.

Upon opening the generated report, the user is presented with the project name, basic information, and an overview of the analyzed metrics and detected code smells. The report then provides a detailed list of all identified code smells, including their descriptions, resolution recommendations, and the specific classes and methods where they were found. Finally, the report contains a comprehensive summary of all the analyzed classes, including their associated metrics, methods, fields, and detected code smells. These details are accessible through interactive toggles, allowing users to drill down into specific areas of interest efficiently.

The proposed metrics were evaluated in a specific Spring boot project with the following parameters:

TABLE II
PROJECT PARAMETERS

Parameter	Value
Number of classes	87
Number of methods	480
LOC	6778
Detected code smells	68

The new metrics were compared with traditional metrics such as Cyclomatic Complexity (CC). While CC provides a basic measure of method complexity, the RCI metric contextualizes this complexity within the class, offering a more actionable insight. For example, methods with moderate CC values but disproportionately high RCI scores were often identified as maintenance hotspots, which traditional metrics might overlook.

The following table shows the results of the evaluation:

TABLE III
COMPARISON OF METRICS

Metric	Number of code smells
Complex method (using only CC)	8
Complex method in class (using RCI)	21
Complex method (using IC)	10

As shown in the table III, the proposed metrics are more effective in identifying code smells than traditional metrics. The RCI metric is especially useful for identifying complex methods within a class, which can be a maintenance hotspot. The IC metric is also effective in identifying classes with high complexity. Using these metrics, developers can detect potential problems sooner and fix them before they become a problem. This can help improve the overall quality of the codebase and reduce maintenance costs.

V. CONCLUSION

Ensuring software quality is essential for maintaining its functionality, reliability, and long-term usability. This study highlights the role of code analysis metrics in addressing complexity and identifying structural weaknesses, commonly referred to as "code smells." The proposed metrics, including

Relative Complexity Index (RCI) and Interaction Complexity (IC), provide enhanced tools to pinpoint potential maintenance issues early in the development process. The successful implementation of these metrics in a Maven plugin for Spring Boot projects demonstrates their applicability and relevance.

Future work could explore extending these metrics to additional programming paradigms and integrating them into continuous integration workflows, fostering a proactive approach to software quality assurance, enhancing to a functional and procedural programming languages, and exploring the potential of these metrics in other programming paradigms. From the implementation side, the plugin could be extended to detect additional code smells and provide more detailed recommendations for fixing them. The user interface could also be improved to make the report more intuitive and user-friendly. For example, the report could include interactive features such as filtering and sorting to help users navigate the information more easily.

ACKNOWLEDGMENT

The work reported here received funding from the EU's NextGenerationEU instrument through the Slovakia's Recovery and Resilience Plan, project No. 09I03-03-V01-00030.

The work reported here was supported by the Cultural and Educational Grant Agency of Slovak Republic (KEGA) under grant No. KG 014STU-4/2024.

REFERENCES

- [1] What is Code Smell How to Avoid it, 2023. Available: <https://www.opsera.io/blog/whatis-code-smell>.
- [2] A. Allard, "31 Code Smells all software engineers must watch out for." Available: <https://pragmaticways.com/31-code-smells-you-must-know/Dispensables>.
- [3] P. Kumar S., "Refactoring 101: Code Smells Bloaters," 2021. Available: <https://medium.com/testvagrant/refactoring-101-code-smells-bloatersf80984859340>.
- [4] A. Allard, "31 Code Smells all software engineers must watch out for." Available: <https://pragmaticways.com/31-code-smells-you-must-know/Abusers>.
- [5] M. Phan, "Refactoring couplers," 2024. Available: <https://ducmanhphan.github.io/2020-01-10-Refactoring-couplers.github.io/2020-01-10-Refactoring-couplers/>.
- [6] "Refactoring Change Preventers in C," 2023. Available: <https://code-maze.com/csharp-refactoring-change-preventers>.
- [7] "Software Unit Test Smells." Available: <https://testsmells.org/>.
- [8] R. Harrison and D. Rodriguez, "An Overview of Object-Oriented Design Metrics," 2001. Available: <https://www.cc.uah.es/drg/b/RodHarRama00.English.pdf>.
- [9] N. Singh, "Size Oriented Metrics in Software Engineering," 2023. Available: <https://www.codingninjas.com/studio/library/size-oriented-metrics-in-software-engineering>.
- [10] R. Ponnala and S. Kumar, "Object Oriented Metrics in Software Development," IX, pp. 119-128, Feb. 2015.
- [11] Y. Kataieva and E. Znava, "Construction of Optimal Geographical Routes Using User-Defined Coordinates Based on Existing Algorithms," in SIST 2023 - 2023 IEEE International Conference on Smart Information Systems and Technologies, Proceedings, pp. 310-313, 2023.
- [12] N. Yehorchenkova, O. Yehorchenkov, M. Finka, V. Ondrejicka, and S. Ondrejickova, "Development of a Conceptual Model for an Information Management System in Spatial Planning Projects. Case Study of Making-City Project," Heliyon, vol. 10, no. 12, 2024.
- [13] J. Perdek and V. Vranic, "Matrix Based Approach to Structural and Semantic Analysis Supporting Software Product Line Evolution", CEUR Workshop Proceedings, 3588, pp. 26-37, 2023.